



操作系统实验

题 目	:	实验7
专业名称	:	计算机科学与技术
授课教师	:	陈鹏飞
学生姓名	:	陈安康
学 号	:	22320021
实验地点	:	实验中心D402
日 期	:	2024/6/1

一、教程中的虚拟内存管理机制与内存分配与释放过程的分析

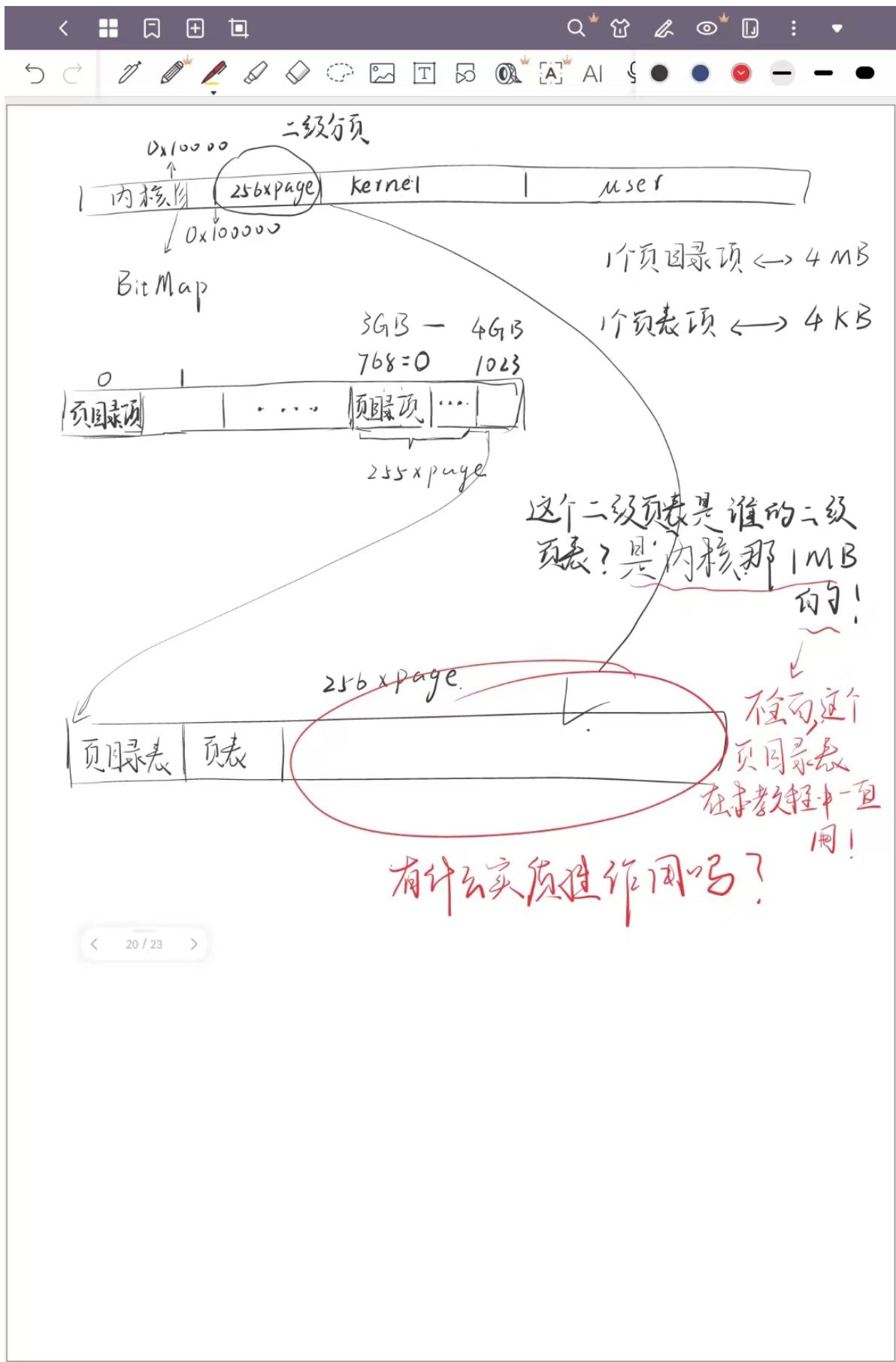
1.位图与地址池实现的分析

首先简单分析一下内存管理的两个基本的结构：位图和地址池的实现。

位图通过一位来代表资源的状态，这样可以用十分小的内存代价来存储内存资源的状态信息。在教程的代码实现上，位图是独立的模块，它只知道位图的初地址和大小，不知道自己标识的资源的具体信息（自己标识的内存资源的地址和每单位资源的大小都不可见），这样使得位图具有更好的可移植性。地址池则是将位图与其标识的资源进行封装，其中的方法本质上都是调用位图的方法通过修改资源位来进行资源的分配和释放。**位图的资源分配释放算法决定了整个内存分配的算法。所以在之后的算法实现上，修改的其实是位图的资源分配与释放方法。**

2.内存结构的分析以及对预留空间实用性的质疑

接着分析教程中所给出的代码所实现的内存结构，如下图：



首先要在开启二级分页之前就将页目录表以及页表建好，并将内核对应的物理地址映射到这个页目录表和页表中，然后将页目录表地址放在cr3寄存器中，因为二级分页机制一旦开启，那么所有的寻址都会按照二级分页来进行，到时候没有页目录表和页表那寻址就会出现错误。因此我们要提前建立好页目录表和页表。除此之外，这个页表放到cr3之后就没有被换下来，在此后的虚拟地址到物理地址的转换等一系列转换，都是用的这个页目录表。

所有的用户进程的3G - 4G实际上都是共享的内核的物理空间，而内核的物理空间和内核的虚拟空间是存在直接映射关系的，所以用户的页目录表只要照抄内核页目录表的3G - 4G即可。（这段是我根据教程解释猜测的，因为没有建立用户进程，教程到最后也没明确展示出这一段共享内存是怎么共享的，所以这里存疑。）

将内核虚拟地址池的起始设置为0xc0100000（首先解释为什么不是从0开始的，虚拟内核的二级页表初始的1MB已经为了保证打开二级分页机制能够正常运行，存放了内核的映射（虚拟地址与物理地址相同），所以如果是从0开始的，则会在寻址时找到内核上去，这是我们不允许的，我们只希望它能访问到划分的内核物理空间上。所以默认起始位置是从大于1MB的地方开始的）教程上说是为了保证所有的进程所用的空间都是共享的那部分空间（为什么不是0xc0000000？因为768号的页目录项与0号页目录项相同，所以会使得在3G - 4G空间的前1MB被认为是存放了内核，因此需要从0xc0100000开始）

可是教程到最后也没给出共享的实质性理由，可能是为了之后的实验做铺垫，因为在我看来，由于共享机制在当前教程中并没有实质性的展现，这样做只会压缩虚拟内存的可以使用的空间。

再来是对预留的254个页表的实用性的质疑。除去装载内核需要预留建立页目录表和页表的那一部分，其余的部分感觉只是用来充数的，丝毫没有任何用处。教程解释是预留该3G-4G这一段共享空间的页表，可问题是，教程之后的实现页表分配时，若页目录项没有对应的页表，分配的物理页是从内核物理池中分配的，而物理池是管不了那预留的256个页的，**所以到头来，新分配的页表最后建在了物理池中，那么空余的254个页表（前两个页面一个装了页目录表一个装了内核页表，有存在价值）的意义是什么呢？**

为了验证我的想法，即那254个页表在**本次实验（仅限于本次实验下！之后的实验中这254个页面可能有实际用途，我不清楚）**中并没有发挥任何作用，在原有的教程代码基础上进行模拟，我在分配了页之后进行写操作，即通过memset去进行页面覆盖，因为只分配不去修改内存的话根本看不出任何变化，如下：

```
char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 1000);
char *p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
char *p3 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);

printf("%x %x %x\n", p1, p2, p3);
memset(p1, 1, PAGE_SIZE * 1000);
memoryManager.releasePages(AddressPoolType::KERNEL, (int)p2, 10);
p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);

printf("%x\n", p2);

p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
```

然后我将预留空间改成2：

```
// 预留的内存
int usedMemory = 2 * PAGE_SIZE + 0x100000;
if (this->totalMemory < usedMemory)
{
    printf("memory is too small, halt.\n");
    asm_halt();
}
// 剩余的内存
int freeMemory = this->totalMemory - usedMemory;
```

因为我改了预留空间，所以内核与用户的物理池管控的空间就会变大，而其他地方没有任何改变，并且1000个页面的分配量会强迫新建一个页表（因为之前的页表剩余大小肯定不够了）。倘若这254个页面真有必要存在（教程中的想法应该是让他们做页表，那么理想中新建的页表会紧挨着装载了内核物理地址的那个页表，但实际代码实现中并没有这样），那么这个覆盖操作会将这254个页面里面的部分页面进行覆盖，必然会导致运行的错误，结果运行如下：

```
9 MemoryManager::MemoryManager()
10 {
11     initialize();
12 }
13
14 void MemoryManager::initialize()
15 {
16     //获得总内存大小
17     this->totalMemory = 0;
18     this->totalMemory = getTotalMemory();
19
20     // 预留的内存
21     int usedMemory = 2 * PAGE_SIZE + 0x100000;
22     if (this->totalMemory < usedMemory)
23     {
24         printf("memory is too small, halt.\n");
25         asm_halt();
26     }
27     // 剩余的空闲的内存
28     int freeMemory = this->totalMemory - usedMemory;
29
30     int freePages = freeMemory / PAGE_SIZE;
31     int kernelPages = freePages / 2;
32     int userPages = freePages - kernelPages;
33
34     int kernelPhysicalStartAddress = usedMemory;
35     int userPhysicalStartAddress = usedMemory + kernelPages * PAGE_SIZE;
36
37     int kernelPhysicalEndAddress = kernelPhysicalStartAddress + kernelPages * PAGE_SIZE;
38     int userPhysicalEndAddress = userPhysicalStartAddress + userPages * PAGE_SIZE;
```

```
Machine View
iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00
Booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x102000
  total pages: 16111 ( 62 MB )
  bitmap start address: 0x10000
user pool
  start address: 0x3FF1000
  total pages: 16111 ( 62 MB )
  bit map start address: 0x107DE
kernel virtual pool
  start address: 0xC0100000
  total pages: 16111 ( 62 MB )
  bit map start address: 0x10FBC
C0100000 C04E8000 C04F2000
C0556000
C04E8000
```

对比一下将预留空间大小改为256个页面的代码的运行结果：

```
{
    initialize();
}

void MemoryManager::initialize()
{
    //获得总内存大小
    this->totalMemory = 0;
    this->totalMemory = getTotalMemory();

    // 预留的内存
    int usedMemory = 256 * PAGE_SIZE + 0x100000;
    if (this->totalMemory < usedMemory)
    {
        printf("memory is too small, halt.\n");
        asm_halt();
    }
    // 剩余的空闲的内存
    int freeMemory = this->totalMemory - usedMemory;

    int freePages = freeMemory / PAGE_SIZE;
    int kernelPages = freePages / 2;
    int userPages = freePages - kernelPages;

    int kernelPhysicalStartAddress = usedMemory;
    int userPhysicalStartAddress = usedMemory + kernelPages * PAGE_SIZE;
}
```

```
Machine View
iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00
Booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0x10000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x10F9C
C0100000 C04E8000 C04F2000
C0556000
C04E8000
```

对比上面两张图可以看到，除了两个地址池的大小和起始地址有变化（因为我改了预留空间大小的缘故），其余都是照常运行的，再来对比一下将预留大小空间改为1个页面时的运行结果（这里我将覆盖数值改为了0，因为改为1之后会出现错误闪退，截不了屏，不过结果都是运行出现了问题）可以看到，运行过程出现了问题，因为这个页表中存的是内核物理地址，没有了预留空间的保护，新分配时将这个页当做普通页分配了出去，最后被冲了，导致错误，与我们预想一致：

```
9 MemoryManager::MemoryManager()
10 {
11     initialize();
12 }
13
14 void MemoryManager::initialize()
15 {
16     //获得总内存大小
17     this->totalMemory = 0;
18     this->totalMemory = getTotalMemory();
19
20     // 预留的内存
21     int usedMemory = 1 * PAGE_SIZE + 0x100000;
22     if (this->totalMemory < usedMemory)
23     {
24         printf("memory is too small, halt.\n");
25         asm_halt();
26     }
27     // 剩余的空闲的内存
28     int freeMemory = this->totalMemory - usedMemory;
29
30     int freePages = freeMemory / PAGE_SIZE;
31     int kernelPages = freePages / 2;
32     int userPages = freePages - kernelPages;
33
34     int kernelPhysicalStartAddress = usedMemory;
35     int userPhysicalStartAddress = usedMemory + kernelPages * PAGE_SIZE;
36
37     int kernelPhysicalEndAddress = kernelPhysicalStartAddress + kernelPages * PAGE_SIZE;
38     int userPhysicalEndAddress = userPhysicalStartAddress + userPages * PAGE_SIZE;
39
40     int kernelPhysicalStartAddress = usedMemory;
41     int userPhysicalStartAddress = usedMemory + kernelPages * PAGE_SIZE;
42
43     int kernelPhysicalEndAddress = kernelPhysicalStartAddress + kernelPages * PAGE_SIZE;
44     int userPhysicalEndAddress = userPhysicalStartAddress + userPages * PAGE_SIZE;
45 }
```

```
Machine View
Unhandled interrupt happened, halt...
iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00
Booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x101000
  total pages: 16111 ( 62 MB )
  bitmap start address: 0x10000
user pool
  start address: 0x3FF0000
  total pages: 16112 ( 62 MB )
  bit map start address: 0x107DE
kernel virtual pool
  start address: 0xC0100000
  total pages: 16112 ( 62 MB )
  bit map start address: 0x10FBC
C0100000 C04E8000 C04F2000
```

由此可以看到，教程中分配的256个页面，真正用时只有前两个页面真正起了作用，后面的页面只是留了个空位置，建立页表时也没有建在上面，在本次实验时毫无作用。

3.页内存分配三步骤的分析：

言归正传，接下来进行对页面分配过程的解释：

首先解释一下教程中的物理页面分配策略：**教程中给定的分配算法采用了first_fit算法来进行分配**。假设要分配x个页面，在物理池中从头到尾寻找是否有x个连续空闲的页，若有直接分配，否则继续寻找，直到找到合适的位置或者找不到合适的位置，进行返回。由于所有的地址池都是用的同样的位图，所以虚拟池和物理池的分配策略都是一样的，下图是教程中的代码：

```
int Bitmap::allocate(const int count)
{
    if (count == 0)
        return -1;
    int index, empty, start;
    index = 0;
    while (index < length)
    {
        // 越过已经分配的资源
        while (index < length && get(index))
            ++index;
        // 不存在连续的count个资源
        if (index == length)
            return -1;
        // 找到1个未分配的资源
        // 检查是否存在从index开始的连续count个资源
        empty = 0;
        start = index;
        while ((index < length) && (!get(index)) && (empty < count))
        {
            ++empty;
            ++index;
        }
        // 存在连续的count个资源
        if (empty == count)
        {
            for (int i = 0; i < count; ++i)
            {
                set(start + i, true);
            }

            return start;
        }
    }
    return -1;
}
```

接下来分析教程中的代码：

```
// 对于物理页，从0号物理页开始，一个一个物理页去判断，直到找到足够大的可以容纳的物理页的空闲物理页的大小。
char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);
char *p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
char *p3 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);

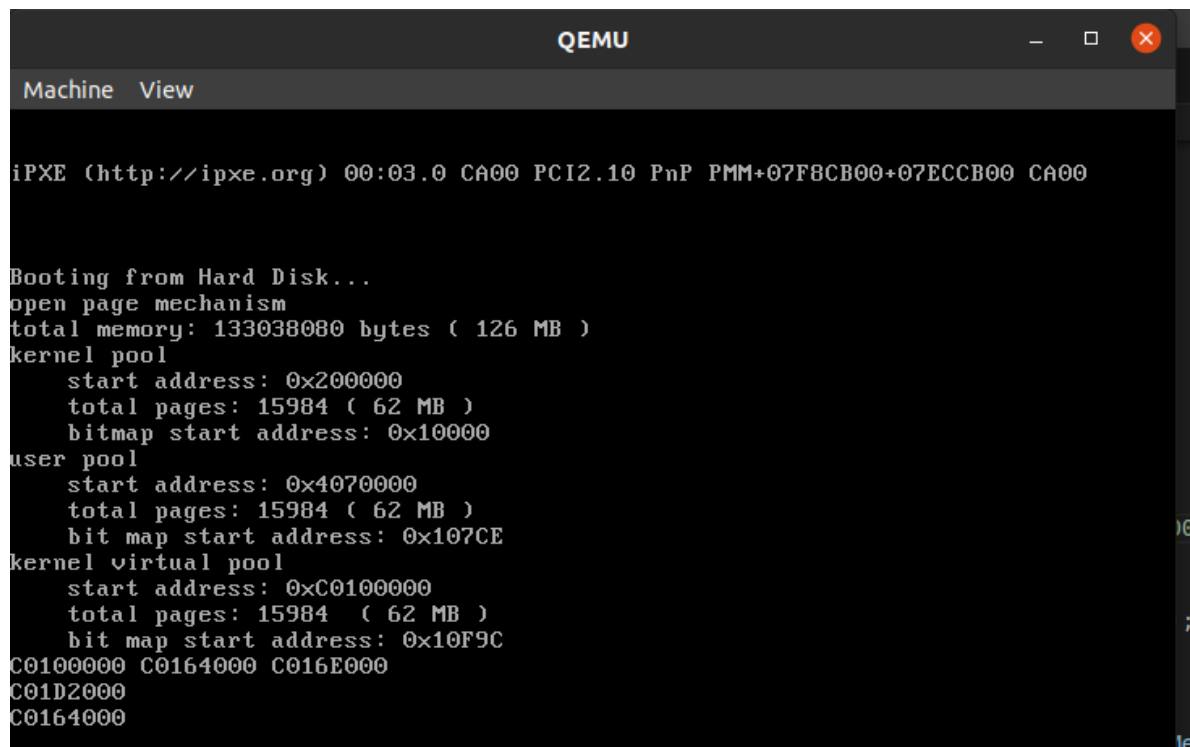
printf("%x %x %x\n", p1, p2, p3);
// memset(p1, 1, PAGE_SIZE * 100);
memoryManager.releasePages(AddressPoolType::KERNEL, (int)p2, 10);
p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 100);

printf("%x\n", p2);

p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);

printf("%x\n", p2);
```


首先接连分配了210个页面，由于用来装内核的页表还有足够的空间来存放这些页表项，所以不会新建页表，然后释放掉10个页面，再申请100个页面，接着再申请10个页面。按照以上的分配逻辑，再申请的100个页面会往后占用空间，而再申请的10个页面会占据释放掉的10个页面的位置，运行结果如下：



```
QEMU
Machine View
iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00

Booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0x10000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x10F9C
C0100000 C0164000 C016E000
C01D2000
C0164000
```

可以看到如我们猜测，首先三个分配所占据的地址是连续的，表明分配是连续的（这个经过简单的计算就能证明）中间没有新建页表，释放完10个页面后，100个页面分配时是紧挨着0xC016E000的100个页面往下，而分配的10个页面是占了原先释放掉的10个页面的位置。

表面的原理都走通了，接下来分析一下背后运行的逻辑：

（ps:下面说的物理地址在某些情况下指的物理页号，请视情况辨别）

分配页面，首先物理页和虚拟页都要存在，在这种情况下，**最重要的就是建立物理页与虚拟页之间的关系**，并将这种关系记录在页目录表与页表中，所以在知道虚拟页虚拟地址与物理页的物理地址的情况下，只要将物理地址写在该虚拟地址对应的页表项中就好了。**注意，所谓的物理地址在已经开启分页机制后就单纯是一个数值的含义，是我们预想中应当存入内存的物理地址（事实也确实如此）。**所以地址池中名义上是记录了程序的物理地址，但我们没有通过地址池直接访问过任何东西，地址池只是负责记录这样的一些预想中应当存入的物理地址对应的值。**只有这些物理地址的数值放入页表中，并且在我们知道其对应的虚拟地址的情况下，我们才能通过虚拟地址访问到这个物理地址存放的数据。**因为我们开启了二级分页，那么物理地址值只有放在了页表中，与某个虚拟地址联系起来，对于我们来说才是能够通过虚拟地址访问到物理地址所指向的内存，其余情况下本质上都只是一个数值而已。下面这个图片展示了我们通过虚拟地址找到了物理地址所指向的那个内容，并且可以对其进行操作，**可以看到我们真的分配了一块内存，而不是玩“文字游戏”。**关键就是我们放入页表的物理地址和虚拟地址关联了起来，通过MMU完成了转换。

通过以上可以知道，关键就是将物理地址写进页表中，那么就要对页表进行操作，修改其页表项，那么由上面的分析，**从我们开启二级分页的那一刻起，我们就只能操作虚拟地址。无法操作物理地址。**因此，我们要知道我们要操作的页目录项与页表项的虚拟地址。这时候就要求我们依据现在的虚拟地址推算其所位于的页目录项页表项的虚拟地址，然后将上面的物理地址填进去，再设置一些控制位即可。

为了推算方便，我们在之前在页目录项的最后一项的地址写入的是页目录表的物理地址（其实是不是最后一项无所谓，只要是在页目录表中约定好的位置就行，一定要在页目录表中，这很重要！），而MMU是从页目录表入手转换的，这样**我们就可以实现“原地打转”的效果。**因为MMU是死的，它才不管数据的具体含义，它是个硬件，只会死板的寻址。

```

50
51 int MemoryManager::toPDE(const int virtualAddress)
52 {
53     return (0xfffff000 + (((virtualAddress & 0xffc00000) >> 22) * 4));
54 }
55
56 int MemoryManager::toPTE(const int virtualAddress)
57 {
58     return (0xffc00000 + ((virtualAddress & 0xffc00000) >> 10) + (((virtualAddress & 0x003ff000) >> 12) * 4));
59 }
60

```

接下来是对转换代码的解释：由于虚拟地址中已经把自己所在的页目录项和页表项告诉你了，那么，对于页目录项，我们最后肯定是在页目录表中找到它，那么最后的偏移量就是页目录项在页目录表中的偏移量，由此往上推，上两个“页目录表和页表”寻址寻到最后寻到了页目录表的头上，那么将上面的值全部设成1111111111，这样两次寻找都会是原地打转，最后页内偏移*4就会寻找到页目录项了。通过这个方法去分析页表项，最后我们寻找的页内偏移是在页表里面的，那么上一层寻址寻到了页表，谁记录了页表的物理地址呢？是页目录表，那么再上一层寻址寻找到了页目录表。以此，只要将前十位置1，接下来10位是虚拟地址高10位，最后的偏移量是虚拟地址21-12位*4。凭借这个方法，我们就巧妙地获得了虚拟地址页目录项与页表项的虚拟地址。

找到页目录项与页表项之后，我们就对其进行修改：如果页目录项没有记载此页表，那么从物理池中分配一个页来建页表，然后进行初始化，页目录表对应项填入页表的物理地址，页表对应项填入需要映射到的物理页的物理地址，然后设置一些控制位即可，完整的建立连接代码如下：

以上是如何将虚拟页与物理页之间通过页表关联起来。除此之外，在申请内存时，**虚拟页是一次性按照位图申请规定好的策略进行申请的，而物理页是一页一页进行申请**，进行与虚拟页的关联的，这样也是为了方便实现与虚拟页的一一对应，代码部分如下：

```

// 第一步：从虚拟地址池中分配若干虚拟页
int virtualAddress = allocateVirtualPages(type, count);
if (!virtualAddress)
{
    return 0;
}

bool flag;
int physicalPageAddress;
int vaddress = virtualAddress;

// 依次为每一个虚拟页指定物理页
for (int i = 0; i < count; ++i, vaddress += PAGE_SIZE)
{
    flag = false;
    // 第二步：从物理地址池中分配一个物理页
    physicalPageAddress = allocatePhysicalPages(type, 1);
    if (physicalPageAddress)
    {
        flag = true;
        // 将物理地址填入页表
        // 将虚拟地址填入页目录表
    }
}

```

申请的本质就是从位图里面查找有没有空闲的资源，通过first_fit算法进行分配，这个对应了页内存分配的第1步与第2步，因为比较简单，只是对位图的资源位进行修改并返回，所以不再详细赘述。

以上就是页内存分配的三个步骤的分析。

4.页内存的释放

如果空闲物理页没有了，那么我们需要将之前分配的虚拟页和物理页都释放掉，这里设计通过虚拟地址找到对应的物理地址（物理页释放需要物理地址），**通过上面分析的方法通过虚拟地址找到页表虚拟地址，在页表里面读出物理地址的页号，然后通过虚拟地址找到页内偏移（物理地址与虚拟地址页内偏移一致）然后拼凑出物理地址**，就可以释放已经与虚拟页建立联系的物理页了。**释放内存的操作本质只是简单的修改位图的资源位，并没有对页内的数据进行清除。**


```
int MemoryManager::vaddr2paddr(int vaddr)
{
    int *pte = (int *)toPTE(vaddr);
    int page = (*pte) & 0xfffff000;
    int offset = vaddr & 0xfff;
    return (page + offset);
}
```

```
// 分配失败，释放前面已经分配的虚拟页和物理页表
if (!flag)
{
    // 前i个页表已经指定了物理页，这些也要释放
    releasePages(type, virtualAddress, i);
    // 剩余的页表未指定物理页
    releaseVirtualPages(type, virtualAddress + i * PAGE_SIZE, count - i);
    return 0;
}
```

以上就是对教程中涉及的内存管理的内存分配释放过程以及一些内存管理数据结构及其功能的分析。

二、best_fit物理页分配算法的实现：

因为教程已经实现了first_fit分配算法，这里我们实现best_fit分配算法。由之前的分析已经知道了，位图的分配算法决定了页内存的分配算法，所以只要修改位图的分配算法就好。这里我们从头到尾遍历位图，找到大于等于所需分配页面大小的连续空闲资源，并且记录下最小的空闲资源，最后修改最小的空闲资源位并且进行返回，代码如下：

```

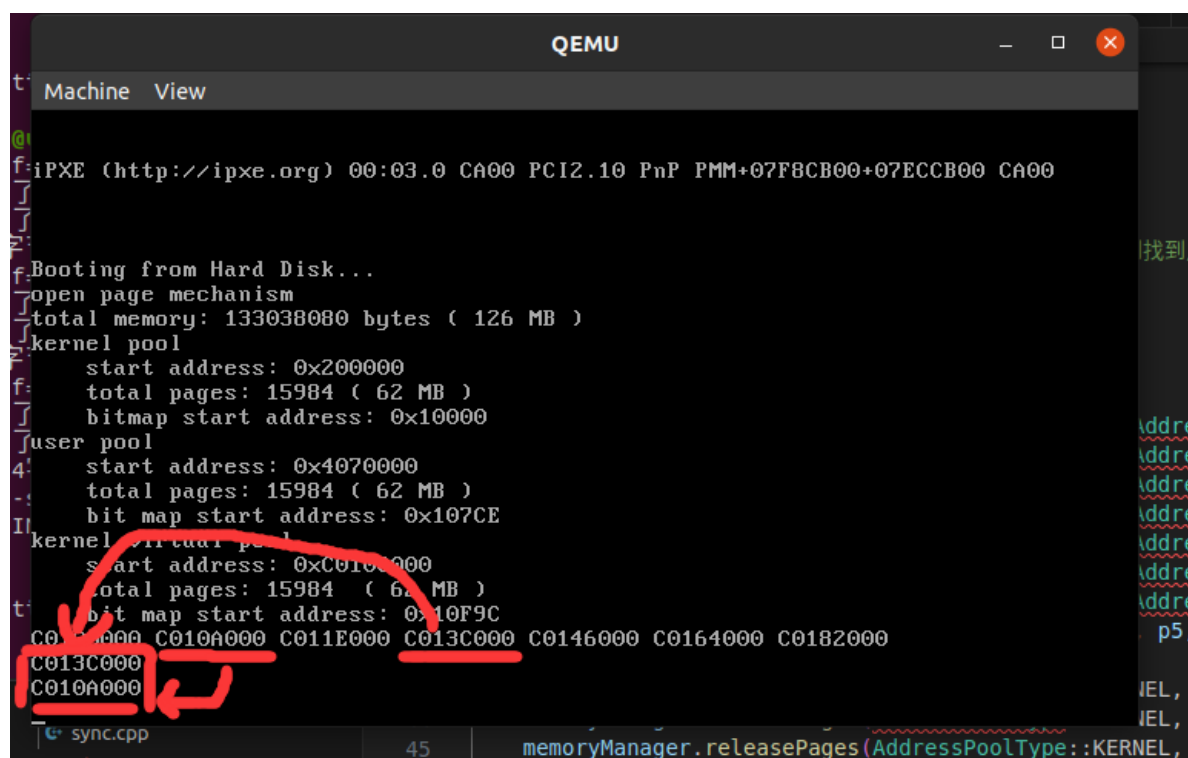
int BitMap::allocate(const int count)
{
    if (count == 0)
        return -1;
    int index, empty, start;
    int best_fit = -1, gap = length;
    index = 0;
    while (index < length)
    {
        // 越过已经分配的资源
        while (index < length && get(index))
            ++index;
        // 不存在连续的count个资源
        if (index == length)
            return -1;
        // 找到1个未分配的资源
        // 检查是否存在从index开始的连续count个资源
        empty = 0;
        start = index;
        while ((index < length) && (!get(index)))
        {
            ++empty;
            ++index;
        }
        if (empty - count >= 0 && empty - count < gap) {
            gap = empty - count;
            best_fit = start;
        }
    }
    if (best_fit != -1)
    {
        for (int i = 0; i < count; ++i)
        {
            set(best_fit + i, true);
        }
    }
    return best_fit;
}

```

测试用例设置如下，我连续分配了7次，分别分配页面数量为10、20、30、10、30、30、30，接着释放掉第二次分配的20页和第四次分配的10页，和第六次分配的30页，然后重新申请10页和15页。若 best_fit 算法有效，则再次申请的10个页面应占据第四次分配的页面的位置，15个页面应占据第二次分配的页面位置。

```
// 这些是测试best_fit物理页置换算法的用例
char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
char *p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 20);
char *p3 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 30);
char *p4 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
char *p5 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 30);
char *p6 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 30);
char *p7 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 30);
printf("%x %x %x %x %x %x %x\n", p1, p2, p3, p4, p5, p6, p7);
// memset(p1, 1, PAGE_SIZE * 10);
memoryManager.releasePages(AddressPoolType::KERNEL, (int)p2, 20);
memoryManager.releasePages(AddressPoolType::KERNEL, (int)p4, 10);
memoryManager.releasePages(AddressPoolType::KERNEL, (int)p6, 30);
p2 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
printf("%x\n", p2);
p4 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 15);
printf("%x\n", p4);
```

运行结果如下，可以看到新分配的页面的首地址分别与第四次分配，第二次分配的页面的首地址一致，算法正确：



```
QEMU
Machine View
f: iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00
f: Booting from Hard Disk...
f: open page mechanism
f: total memory: 133038080 bytes ( 126 MB )
f: kernel pool
f:   start address: 0x200000
f:   total pages: 15984 ( 62 MB )
f:   bitmap start address: 0x10000
f: user pool
f:   start address: 0x4070000
f:   total pages: 15984 ( 62 MB )
f:   bit map start address: 0x107CE
f: kernel virtual pool
f:   start address: 0xC010A000
f:   total pages: 15984 ( 62 MB )
f:   bit map start address: 0x10F9C
f: C010A000 C010A000 C011E000 C013C000 C0146000 C0164000 C0182000
f: C013C000
f: C010A000
f: sync.cpp 45 memoryManager.releasePages(AddressPoolType::KERNEL,
```

三、LRU页面置换算法：

因为真正的页面置换要从磁盘读取页到内存中，而我们建立的页表目前所有的操作都是在内存中，如果要和磁盘交互，那么需要实现将页先放到磁盘中，再调上来进行置换。虽然前面已经实现过从磁盘上加载mbr到内存中，但是如果真正实现磁盘上的页面替换，对我来说工程量太大且难以完成。首先，MMU在地址上访问不到页会引发缺页错误，执行缺页中断，这时需要我们实现缺页故障的中断处理程序，在中断处理程序中，我们要完成页面在磁盘中的寻址，这个部分要求我们从页表或者辅助数据结构（如交换区表）中获取页面在磁盘上的位置。然后如果内存满的话，将其他的页面通过页面置换算法置换下来放到磁盘，将需要的页面读入内存中。其中涉及到的定位页面在磁盘中的位置以及一些其他步骤的实现细节（如缺页故障处理程序的实现注意的细节）太多，并且这个工程要求对硬件还要有一定的了解。工作量太大，要求知道的细节太多，我感觉自己短时间内无法胜任。

1.模拟环境的构建

因此，为了达到实验目的，我决定在内存中完成对算法流程的模拟。所有的操作全部放在内存中。对每个进程（这里我把它看做进程了），维护一个固定大小的内存队列（存放在PCB中），里面存放进程最近的引用的页面的页面号，设计一个函数模拟引用操作，如果引用的页面不在队列中，触发“页面置换”，页面置换采用LRU算法。因为没有与硬盘的交互了，一些流程进行了简化。除此之外，我还增加了判断功能，来判断当前页面号是否为非法，来保证访问操作的正确性。

本来想着独立地设计一个数据结构，但是由于我们没有实现new内存分配，所以意味着要规定存放的内存位置，这有些麻烦，所以索性直接放在PCB里面，这样也不用为内存规划发愁了。

代码实现上，因为该数据结构保存在PCB中，所有需要在PCB中添加相关的内容，如下：

```
typedef struct Node {
    //虚拟页号
    int address;
    //LRU
    int lru;
} Node;
class VirtualPage_Queue {
    //该线程现有的内存队列
    Node queue[MEMORY_QUEUE_SIZE];
    //队列的大小
    int size;
public:
    VirtualPage_Queue() {
        initialize();
    }
    void initialize() {
        size = MEMORY_QUEUE_SIZE;
        for(int i = 0; i < size; ++i) {
            queue[i].address = 0;
            queue[i].lru = 0;
        }
    }
    void swap(int index, int addr) {
        queue[index].address = addr;
        queue[index].lru = 0;
    }
}
```

```

void reference(int addr) {
    //判断是否访问非法虚拟页
    int *pte = (int *)toPTE_thread(addr);
    if(!(*pte & 0x00000001)) {
        printf("reference error!!!\n");
        return;
    }
    //LRU算法
    int max = 0, index;
    for(int i = 0; i < size; ++i) {
        if(queue[i].address == addr) {
            queue[i].lru = 0;
            return;
        }
        ++queue[i].lru;
        if(queue[i].lru > max) {
            max = queue[i].lru;
            index = i;
        }
    }
    swap(index, addr);
}

void print() {
    printf("queue : ");
    for(int i = 0; i < size; ++i) {
        printf("%x ", queue[i].address);
    }
    printf("\n");
}

int getsize() {
    return size;
}

int toPTE_thread(const int virtualAddress)
{
    return (0xffc00000 + ((virtualAddress & 0xffc00000) >> 10) + (((virtualAddress & 0x003ff000) >> 12) * 4));
}

```

在此数据结构中，定义了一个队列，大小设置为3，每一项存了虚拟页号（指的虚拟地址高20位）以及LRU位。

结构中定义了一个方法reference来模拟引用页面，内容也很简单，只是判断是否访问非法虚拟页，然后实现队列更新。在里面实现了LRU算法的基本逻辑：

在一次搜寻中，对于队列中不是目标页面的页面，LRU位加1。

如果访问的页面在队列里面，设置其LRU位为0。

如果都不是，则将LRU值最大的页面换出，换成目标页面，并将目标页面LRU置0。

通过以上算法就可以实现LRU算法，保证换出去的页面永远是最近访问次数最少的。

在PCB中添加队列实例，然后要在创建线程的函数中显示调用初始化函数进行初始化：

```

struct PCB
{
    int *stack; // 栈指针，用于调度时保存esp
    char name[MAX_PROGRAM_NAME + 1]; // 线程名
    enum ProgramStatus status; // 线程的状态
    int priority; // 线程优先级
    int pid; // 线程pid
    int ticks; // 线程时间片总时间
    int ticksPassedBy; // 线程已执行时间
    ListItem tagInGeneralList; // 线程队列标识
    ListItem tagInAllList; // 线程队列标识
    VirtualPage_Queue memory_queue; // 页面置换队列
};

```

```

memset(thread, 0, PCB_SIZE);

for (int i = 0; i < MAX_PROGRAM_NAME && name[i]; ++i)
{
    thread->name[i] = name[i];
}

thread->status = ProgramStatus::READY;
thread->priority = priority;
thread->ticks = priority * 10;
thread->ticksPassedby = 0;
thread->pid = ((int)thread - (int)PCB_SET) / PCB_SIZE;

thread -> memory_queue.initialize();

// 线程栈
thread->stack = (int *)((int)thread + PCB_SIZE);
thread->stack -= 7;
thread->stack[0] = 0;
thread->stack[1] = 0;
thread->stack[2] = 0;
thread->stack[3] = 0;
thread->stack[4] = (int)function;
thread->stack[5] = (int)program_exit;
thread->stack[6] = (int)parameter;

allPrograms.push_back(&(thread->tagInAllList));
readyPrograms.push_back(&(thread->tagInGeneralList));

// 恢复中断
interruptManager.setInterruptStatus(status);

```

这样，我们的模拟环境就创建好了。

2.测试用例

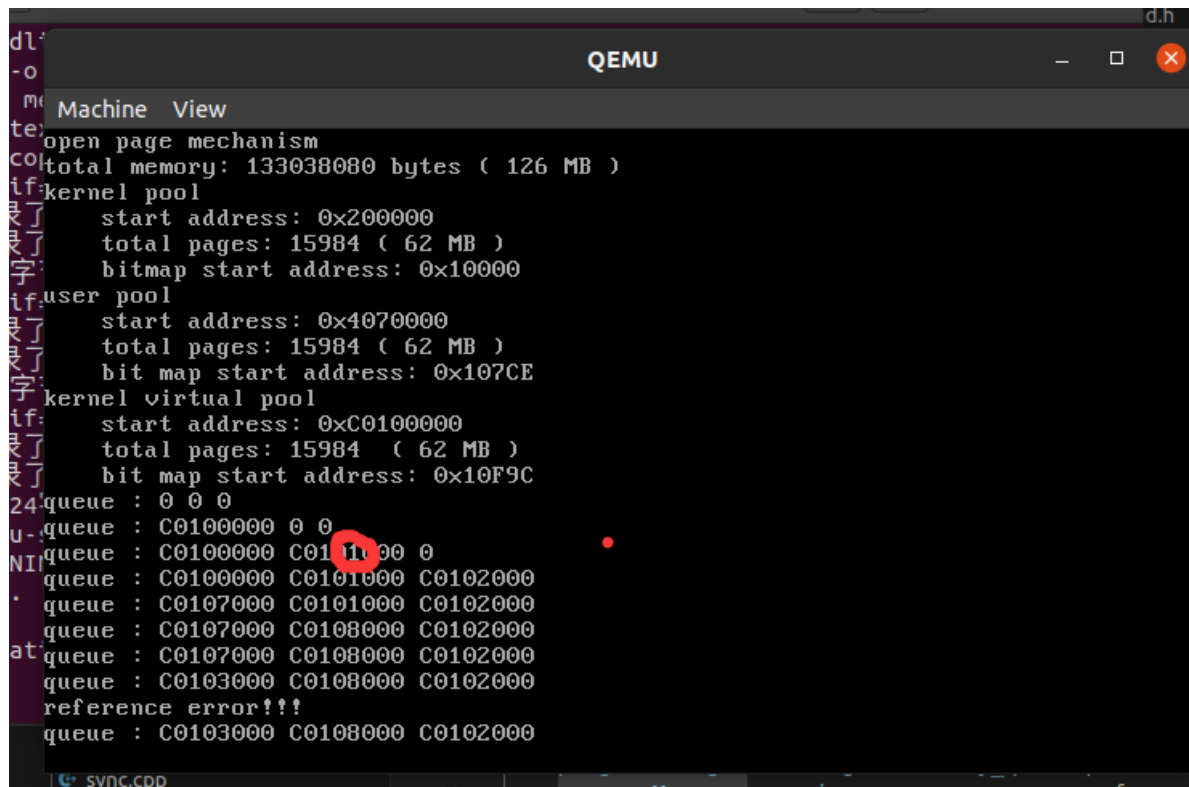
测试用例上，我们在线程中申请了10个页面，然后按照0 1 2 7 8 2 3 10（10不在申请的页面中，所以预期是会报访问错误信息）的顺序来进行访问，预期情况下，前三次都会发生替换操作，然后访问7、8页面时会分别替换掉0、1页面，2页面访问命中不会替换，3会替换掉7，最后的10访问非法报错，不改变队列。

```

//这些是页面置换算法的测试用例
char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 10);
programManager.running -> memory_queue.print();
programManager.running -> memory_queue.reference((int)(p1));
programManager.running -> memory_queue.print();
programManager.running -> memory_queue.reference((int)(p1) + PAGE_SIZE);
programManager.running -> memory_queue.print();
programManager.running -> memory_queue.reference((int)(p1) + 2 * PAGE_SIZE);
programManager.running -> memory_queue.print();
programManager.running -> memory_queue.reference((int)(p1) + 7 * PAGE_SIZE);
programManager.running -> memory_queue.print();
programManager.running -> memory_queue.reference((int)(p1) + 8 * PAGE_SIZE);
programManager.running -> memory_queue.print();
programManager.running -> memory_queue.reference((int)(p1) + 2 * PAGE_SIZE);
programManager.running -> memory_queue.print();
programManager.running -> memory_queue.reference((int)(p1) + 3 * PAGE_SIZE);
programManager.running -> memory_queue.print();
programManager.running -> memory_queue.reference((int)(p1) + 10 * PAGE_SIZE);
programManager.running -> memory_queue.print();

```


运行结果如下：



```
Machine View
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
  start address: 0x200000
  total pages: 15984 ( 62 MB )
  bitmap start address: 0x10000
user pool
  start address: 0x4070000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x107CE
kernel virtual pool
  start address: 0xC0100000
  total pages: 15984 ( 62 MB )
  bit map start address: 0x10F9C
queue : 0 0 0
queue : C0100000 0 0
queue : C0100000 C0111000 0
queue : C0100000 C0101000 C0102000
queue : C0107000 C0101000 C0102000
queue : C0107000 C0108000 C0102000
queue : C0107000 C0108000 C0102000
queue : C0103000 C0108000 C0102000
reference error!!!
queue : C0103000 C0108000 C0102000
```

上面说的序号对应的就是红色圈圈出来的位，可以看到，结果如我们预想的一样，证明LRU算法实现正确。

四、虚拟内存管理中的BUG：

1.问题展示:

因为内存分配没有进行任何线程同步的预防，因此极有可能发生竞争，考虑一下一种情况：

当多个线程同时调用申请内存的指令时，如果在一个线程申请页操作（可能是物理页也可能是虚拟页）中途被强制进行调度，比如进入了位图中的set（）函数阶段但还未执行完毕，此时另一个线程被换上来，他看这个地址池时，因为上一个线程没有完成set()操作，那么这个线程就会认为这个资源还没有被占用，则它就会占用这个资源，等到切换到另一个线程时，它也申请了这个资源，此时就会出现问

由于这个触发条件有点苛刻，必须恰好在某种特定的时刻才会触发，为了实现这个触发，我在位图的set()函数中设置了延时器，这样触发就会简单不少。

```

// 设置已用
void BitMap::set(const int index, const bool status)
{
    int pos = index / 8;
    int offset = index % 8;
    int c = 0xffffffff;
    while(c) {
        c--;
    }
    // 清0
    bitmap[pos] = bitmap[pos] & ~(1 << offset);

    // 置1
    if (status)
    {
        bitmap[pos] = bitmap[pos] | (1 << offset);
    }
}

```

接下来在主线程里面创建另一个线程（这个设置运行时间长一些），两个各自申请内存，然后打印虚拟地址和物理地址：

```

// programManager.running -> memory_queue.print();
int pid = programManager.executeThread(second_thread, nullptr, "second thread", 10);
if (pid == -1)
{
    printf("can not execute thread\n");
    asm_halt();
}
// while(true) {
    char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 1);
    printf("%x\n", p1);
    printf("%x\n", memoryManager.vaddr2paddr((int)p1));
// }

```

```

void second_thread(void *arg) {
    char *p1 = (char *)memoryManager.allocatePages(AddressPoolType::KERNEL, 1);
    printf("%x\n", p1);
    printf("%x\n", memoryManager.vaddr2paddr((int)p1));
}

```

运行，结果如下：

```
QEMU
Machine View
iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00

Booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
    start address: 0x200000
    total pages: 15984 ( 62 MB )
    bitmap start address: 0x10000
user pool
    start address: 0x4070000
    total pages: 15984 ( 62 MB )
    bit map start address: 0x107CE
kernel virtual pool
    start address: 0xC0100000
    total pages: 15984 ( 62 MB )
    bit map start address: 0x10F9C
C0100000
200000
C0100000
201000
```

可以看到，两个线程申请的虚拟页相同，物理页不同，发生了错误。

2.解决方案：

解决方法很简单，只要解决线程同步问题就可以，在内存管理操作上添加一个信号量，然后在分配开始以及结束时进行P和V操作即可：

```
class MemoryManager
{
public:
    // 可管理的内存容量
    int totalMemory;
    // 内核物理地址池
    AddressPool kernelPhysical;
    // 用户物理地址池
    AddressPool userPhysical;
    // 内核虚拟地址池
    AddressPool kernelVirtual;
    // 信号量
    Semaphore semaphore;

    void MemoryManager::initialize()
    {
        semaphore.initialize(1);
        //获得总内存大小
        this->totalMemory = 0;
        this->totalMemory = getTotalMemory();

        int MemoryManager::allocatePages(enum AddressPoolType type, const int count)
        {
            semaphore.P();
            // 第一步：从虚拟地址池中分配若干虚拟页
            int virtualAddress = allocateVirtualPages(type, count);
            if (!virtualAddress)
```

```

// 分配失败，释放前面已经分配的虚拟页和物理页表
if (!flag)
{
    // 前i个页表已经指定了物理页，这些也要释放
    releasePages(type, virtualAddress, i);
    // 剩余的页表未指定物理页
    releaseVirtualPages(type, virtualAddress + i * PAGE_SIZE, count - i);
    return 0;
}
}
semaphore.V();
return virtualAddress;

```

再次执行，结果如下：

```

QEMU
Machine View
iPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CB00+07ECCB00 CA00
booting from Hard Disk...
open page mechanism
total memory: 133038080 bytes ( 126 MB )
kernel pool
    start address: 0x200000
    total pages: 15984 ( 62 MB )
    bitmap start address: 0x10000
user pool
    start address: 0x4070000
    total pages: 15984 ( 62 MB )
    bit map start address: 0x107CE
kernel virtual pool
    start address: 0xC0100000
    total pages: 15984 ( 62 MB )
    bit map start address: 0x10F9C
C0100000
200000
C0101000
201000
sync.cpp 205 // 剩余的页表未指定物理页

```

执行结果正确。

注意事项

所有的实现代码都在文件夹src下。